

Package ‘mcstprules’

April 29, 2026

Type Package

Title Cost Allocation Rules in Minimum Cost Spanning Tree Problems

Version 0.1.0

Description Computes cost allocation rules for Minimum Cost Spanning Tree Problems (MCSTP). The package implements both algorithmic methods and rules based on cooperative game theory.

License GPL (>= 3)

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Imports gtools,
igraph

URL <https://github.com/luciasouto13/mcstprules>, <https://luciasouto13.github.io/mcstprules>

BugReports <https://github.com/luciasouto13/mcstprules/issues>

Suggests knitr,
rmarkdown

VignetteBuilder knitr

Contents

alloc_rules	2
bird_rule	3
boruvka_rule	5
cc_game	7
conewise_rule	9
dk_rule	12
folk_rule	14
irred_game	15
opt_game	18
ows_rule	20
private_game	22
public_game	24
pws_rule	26

Index	29
--------------	-----------

Description

This page provides an overview of the cost allocation rules implemented in this package for minimum cost spanning tree problems (MCSTP). The rules are organized into two main approaches: those defined directly through algorithms and decomposition techniques, and those defined through the analysis of associated cooperative games.

Details

The cost allocation rules can be classified as follows:

RULES DEFINED THROUGH THE PROBLEM

These rules are obtained directly from the network's structure and classical MST algorithms.

Based on Prim's algorithm:

These rules follow the growth of a single tree starting from the source.

- **bird_rule**: agents sequentially connect to the growing tree, paying the cost of the arc through which they connect to the tree.
- **dk_rule**: a modification of Bird's rule that introduces a pivotal switch in the allocation cost at each step to ensure cost monotonicity.

Based on Kruskal's algorithm:

Based on obligation functions, these rules follow Kruskal's logic of merging connected components using the cheapest available arcs.

- **folk_rule**: the agents divide the cost of connecting their components equally (ERO rule).
- **ows_rule**: optimistic weighted Shapley rule. A generalization of the folk rule that distributes costs proportionally based on asymmetric positive weights.
- **pws_rule**: pessimistic weighted Shapley rule. An alternative weighted extension based on a permutation-driven obligation function.

Based on Boruvka's algorithm:

- **boruvka_rule**: the agents pay the maximum possible proportion of the cheapest arc selected by their component. The numerical outcome is mathematically equivalent to the **folk_rule**.

Based on a cone-wise decomposition:

- **conewise_rule**: a technique that decomposes any general MCSTP into a nonnegative combination of elementary problems (where costs are 0 or 1). It allows extending rules like "folk", "ows", or the Bogomolnaia and Moulin family ("bogomolnaia") to general networks.

RULES DEFINED THROUGH COOPERATIVE GAMES

These rules are defined by first associating a cooperative game (in characteristic function form) to the MCSTP and then applying a solution concept (like the Shapley value or the Nucleolus).

Private game:

- `private_game`: defines the cost of a coalition as the MST cost of the subproblem formed by its members and the source (pessimistic approach).

Irreducible game:

- `irred_game`: based on the irreducible matrix $\bar{C}$, where costs reflect the cheapest way to connect each pair of nodes through the network.

Optimistic game:

- `opt_game`: allows a coalition to connect through nodes outside the coalition at no cost.

Public game:

- `public_game`: assigns to each coalition the minimum cost achievable when connections through other agents are publicly available.

Cycle-complete game:

- `cc_game`: based on the cycle-complete matrix C^* , which induces a concave cooperative game.

Note

When the optimal tree is not unique, rules based on Prim's algorithm compute the symmetric average allocation over all possible permutations. In contrast, rules based on Kruskal's and Boruvka's algorithms, as well as the cone-wise decomposition, are unaffected by tie-breaking choices.

For game-theoretic rules, the functions in this package compute the Shapley value of the associated cooperative game.

References

Bergantiños, G., & Vidal-Puga, J. (2021). A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100.

bird_rule

Bird's Rule for MCSTP

Description

This function computes Bird's cost allocation (Bird, 1976) for a minimum cost spanning tree problem (N_0, C) . The Bird rule, denoted as $B(N_0, C)$, assigns to each agent the cost of the arc that connects them to the source during the construction of a minimum cost spanning tree using Prim's algorithm.

Usage

```
bird_rule(C, draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

<code>C</code>	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
<code>draw</code>	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node). For $n \leq 3$ with ties, it also displays all possible trees.
<code>which</code>	numeric vector indicating which plots to display (only for $n \leq 3$ with ties). If 1, displays the detailed breakdown of all trees and allocations according to the agents' entry orders; if 2, the average allocation is plotted. Default is <code>c(1, 2)</code> .
<code>titles</code>	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

For a given tree g^n constructed following Prim's algorithm, the allocation for each agent $i \in N$ is given by:

$$B_i(N_0, C) = c_{i^0 i},$$

where i^0 is the node to which agent i is first connected in the process of building the network.

When the minimum cost spanning tree is not unique, the rule is extended by averaging the allocations over all possible permutations $\pi \in \Pi_N$:

$$B(N_0, C) = \frac{1}{n!} \sum_{\pi \in \Pi_N} B^\pi(N_0, C),$$

where $B^\pi(N_0, C)$ is the allocation obtained by applying Prim's algorithm to (N_0, C) and solving indifferences by selecting the first agent given by the permutation π , as proposed by Dutta and Kar (2004).

Value

A list containing:

- `bird`: the Bird's allocation vector $B(N_0, C)$ for the natural ordering.
- `arcs`: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- `e_bird`: the extended Bird's allocation (average over permutations).
- `total`: the total cost of the MCST, $m(N_0, C)$.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: a ranking of agents by cost (from highest to lowest; ties marked with *).
- `nperms`: the number of permutations $n!$ computed for the average allocation (if ties exist).
- `perms`: a data frame with the detailed allocations for all permutations (if ties exist).
- `is_unique`: logical; if TRUE, the MCST is unique.

Note

The function identifies if ties exist in the cost matrix. If the tree is unique, the standard Bird rule is returned. If ties are detected, it calculates the allocations for all $n!$ permutations to provide the symmetrical average allocation, as discussed in Bergantiños and Vidal-Puga (2021).

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335–350.

Dutta B, Kar A (2004) Cost monotonicity, consistency and minimum cost spanning tree games. *Games Econom Behav* 48(2):223–248.

See Also

[dk_rule](#) for other rule based on Prim’s algorithm.

[irred_game](#) for a cooperative game whose Shapley value coincides with Bird’s solution when applied to the irreducible matrix $\bar{C}$.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
bird_rule(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
bird_rule(C_inf)

# Matrix input with ties
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
bird_rule(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

boruvka_rule

Boruvka’s Rule for MCSTP

Description

This function computes Boruvka’s cost allocation (Bergantiños and Vidal- Puga, 2011) for a minimum cost spanning tree problem (N_0, C) . Boruvka’s rule, denoted as $\beta^\pi(N_0, C)$, is defined through Boruvka’s algorithm.

Usage

```
boruvka_rule(C, draw = FALSE, titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

The Boruvka allocation divides the cost of the optimal tree through a step-by-step process over γ stages. At each stage s , agents pay the maximum possible proportion p^s of the cheapest arc associated with their connected component.

Let A^s be the set of non-completely paid arcs, ϱ_{ij}^s the proportion of the cost of arc (i, j) already paid, and N_{ij}^s the set of agents paying for arc (i, j) in stage s . The proportion p^s is given by:

$$p^s = \min \left\{ \frac{1 - \varrho_{ij}^{s-1}}{|N_{ij}^s|} : (i, j) \in A^{s-1}, N_{ij}^s \neq \emptyset \right\}.$$

Let a_i^s be the arc partially paid by agent i in stage s . The cost paid by agent i in this stage is:

$$f_i^s = p^s c_{a_i^s}.$$

The process finishes in γ stages when the tree is completely paid, i.e., $\sum_{s=1}^{\gamma} p^s = 1$. The Boruvka allocation for each agent $i \in N$ is given by:

$$\beta_i^\pi(N_0, C) = \sum_{s=1}^{\gamma} f_i^s.$$

Value

A list containing:

- boruvka: the Boruvka's allocation vector $\beta^\pi(N_0, C)$.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- is_unique: logical; if TRUE, the MCST is unique.

Note

Boruvka's allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Boruvka's algorithm yields the same cost distribution.

Furthermore, as demonstrated by Bergantiños and Vidal-Puga (2011), for any order π used in the algorithm, Boruvka's allocation is equivalent to the [folk_rule](#).

References

Bergantiños G, Vidal-Puga J (2011) The folk solution and Boruvka's algorithm in minimum cost spanning tree problems. *Discrete Appl Math* 159(12):1279–1283

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[folk_rule](#) for an equivalent rule based on Kruskal's algorithm.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
boruvka_rule(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
boruvka_rule(C_inf)

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
boruvka_rule(C_mat, draw = TRUE, titles = FALSE)
```

cc_game

Cost Allocations based on the Cycle-complete Game for MCSTP

Description

This function computes the cost allocation associated with the cycle-complete game (Trudeau, 2012) for a minimum cost spanning tree problem (N_0, C) . The cycle-complete game, denoted as (N, v_C^c) (or simply (N, v^c)), is defined as the private game associated with the cycle-complete network (N_0, C^*) . When the Shapley value is applied as the solution concept to this game, the resulting allocation is called *cycle-complete solution*.

Usage

```
cc_game(C, sol = "shapley", draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
sol	character; the solution concept to apply to the game. Currently, only sol = "shapley" (default) is supported.

draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$, it also displays the optimal network for each subcoalition $S \subset N$.
which	numeric vector indicating which plots to display. If 1 (only for $n \leq 3$), displays the optimal network for each subcoalition $S \subset N$; if 2, the allocation for N is plotted. Default is <code>c(1, 2)</code> .
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept used and the total network cost.

Details

The cycle-complete game (N, v_C^c) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^c(S) = m(S_0, C^*),$$

where $m(S_0, C^*)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, evaluated over the cycle-complete costs C^* . The cycle-complete cost is defined as the minimum between the direct cost c_{ij} and the smallest value of the maximum arc cost over all cycles containing i and j , i.e., $c_{ij}^* = \min(c_{ij}, \min_{f \in \mathcal{C}(i,j)} \max_{e \in f} c_e)$, where $\mathcal{C}(i, j)$ denotes the set of all cycles that include both i and j .

For computational efficiency, C^* is obtained via the irreducible matrices $\bar{C}$ of the subgraphs as:

$$c_{ij}^* = \max_{k \in N \setminus \{i,j\}} \bar{c}_{ij}^{N \setminus \{k\}} \text{ for } i, j \in N,$$

and

$$c_{0i}^* = \max_{k \in N \setminus \{i\}} \bar{c}_{0i}^{N \setminus \{k\}} \text{ for } i \in N,$$

where $\bar{C}^{N \setminus \{k\}}$ is the irreducible cost matrix of the network excluding agent k . If no agents can be removed, the irreducible cost of the full network is used.

Applying the Shapley value to this TU game, known as the *cycle-complete solution*, assigns to each agent $i \in N$ the average of their marginal contributions over all possible permutations $\pi \in \Pi_N$:

$$Sh_i(N, v^c) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v^c(Pre(i, \pi) \cup \{i\}) - v^c(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$ ordered by cardinality.
- **C_cc**: the cycle-complete cost matrix C^* .
- **v_c**: a vector containing the characteristic function values $v^c(S)$ for all coalitions.
- **contributions**: a data frame with the marginal contributions for all permutations $\pi \in \Pi_N$.

- shapley: the Shapley value allocation vector (*cycle-complete solution*).
- total: the total cost of the network, $v^c(N)$.
- nperms: the number of permutations $n!$ used to compute marginal contributions and the resulting Shapley value.
- percentage: the share of the total cost allocated to each agent.
- ranking: ranking of agents by cost (from highest to lowest; ties marked with *).

Note

The cycle-complete game is concave, which implies that its Shapley value (the *cycle-complete solution*) belongs to its core. Moreover, $\text{core}(N, v^c) \subseteq \text{core}(N, v^p)$.

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Trudeau C (2012) A new stable and more responsible cost sharing solution for mcst problems. *Games Econom Behav* 75(1):402–412

See Also

- [private_game](#) for the pessimistic game based on the original costs.
- [irred_game](#) for the pessimistic game based on the irreducible cost matrix $\bar{C}$.
- [alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
cc_game(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
cc_game(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
cc_game(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

Description

This function computes cost allocations for minimum cost spanning tree problems (N_0, C) using the cone-wise decomposition introduced by Norde et al. (2004). This technique allows extending rules defined for elementary MCSTPs to any MCSTP.

Usage

```
conewise_rule(C, rule = "folk", weights = NULL, lambda = 1)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
rule	a character string indicating the rule R to be applied to the elementary MCSTPs. One of "folk" (default), "ows", or "bogomolnaia".
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents; only if rule = "ows".
lambda	a non-negative numeric parameter (1 by default); only if rule = "bogomolnaia".

Details

Any MCSTP can be written as a nonnegative combination of elementary MCST $C = \sum_{q=1}^{m(C)} x^q C^q$, where the costs of the arcs in C^q are 0 or 1. A rule R is extended as:

$$R(N_0, C) = \sum_{q=1}^{m(C)} x^q R(N_0, C^q).$$

Let g^q be the network at stage $q \in \{1, \dots, m(C)\}$, and $S_i^q := S(P(g^q), i)$ the connected component containing agent i in the graph induced by zero-cost arcs. For each elementary problem (N_0, C^q) , the agents in a component S not containing the source (0) divide the cost of connecting to the source according to the chosen rule:

- Folk rule ("folk"):

$$F(N_0, C^q) = \begin{cases} \frac{1}{|S_i^q|} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise.} \end{cases}$$

- Optimistic weighted Shapley rule ("ows"):

$$f^{ow}(N_0, C^q) = \begin{cases} \frac{w_i}{\sum_{j \in S_i^q} w_j} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise.} \end{cases}$$

- Bogomolnaia and Moulin family ("bogomolnaia"):

For $\lambda \in [0, +\infty)$,

$$R^\lambda(N_0, C^q) = \begin{cases} \frac{\lambda^{\delta_i}}{\sum_{j \in S_i^q} \lambda^{\delta_j}} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise,} \end{cases}$$

where δ_i denotes the number of non-null arcs in C^q containing agent i .

For $\lambda = +\infty$,

$$R^\lambda(N_0, C^q) = \begin{cases} \arg \max_{j \in S_i^q} \delta_j & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise,} \end{cases}$$

i.e., agents attaining the maximum value of δ_j share the cost equally.

Value

A list containing:

- allocations: the cost allocation vector $R(N_0, C)$ for the chosen rule R .
- rule: the rule R applied for the allocation.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- decomposition: a matrix showing the allocation at each cost level of the decomposition.
- weights: the weight vector used for the allocation; only if rule = "ows".
- lambda: the lambda value used for the allocation; only if rule = "bogomolnaia".

Note

This function focuses on the analytical and numerical decomposition of the cost allocation. For graphical representations, use the direct rule functions (e.g., [folk_rule](#)).

The allocation for rule = "folk" is also obtained when all weights are equal in rule = "ows", or when $\lambda = 1$ in rule = "bogomolnaia".

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Norde H, Moretti S, Tijs S (2004) Minimum cost spanning tree games and population monotonic allocation schemes. *Eur J Oper Res* 154(1):84–97

See Also

[folk_rule](#), [ows_rule](#) for direct implementations of these rules.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
C <- c(12, 15, 20, 4, 6, 8)

# Folk rule
conewise_rule(C, rule = "folk")

# Optimistic weighted Shapley rule
conewise_rule(C, rule = "ows", weights = c(2, 1, 1))

# Bogomolnaia and Moulin family
conewise_rule(C, rule = "bogomolnaia", lambda = 2)
conewise_rule(C, rule = "bogomolnaia", lambda = Inf)
```

dk_rule

*Dutta-Kar's Rule for MCSTP***Description**

This function computes Dutta-Kar's cost allocation (Dutta and Kar, 2004) for a minimum cost spanning tree problem (N_0, C) . The Dutta-Kar rule, denoted as $DK(N_0, C)$, is defined through Prim's algorithm with a pivotal switch in the allocation cost at each step to satisfy cost monotonicity.

Usage

```
dk_rule(C, draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node). For $n \leq 3$ with ties, it also displays all possible trees.
which	numeric vector indicating which plots to display (only for $n \leq 3$ with ties). If 1, displays the detailed breakdown of all trees and allocations according to the agents' entry orders; if 2, the average allocation is plotted. Default is c(1, 2).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Prim's algorithm, let $x^0 = 0$. At each stage $p \in \{1, \dots, n\}$, the algorithm selects the minimum cost arc (i^p, j^p) that connects a new agent j^p to the already connected set of nodes. We update the accumulated cost as:

$$x^p = \max\{x^{p-1}, c_{i^p j^p}\}.$$

For each agent $i \in N$, there exists a stage $p(i)$ such that $i = j^{p(i)}$. Dutta-Kar's allocation for agent $i \in N$ is defined as:

$$DK_i(N_0, C) = \min\{x^{p(i)-1}, c_{i^{p(i)} j^{p(i)}}\}.$$

When the minimum cost spanning tree is not unique, the rule is extended by averaging the allocations over all possible permutations $\pi \in \Pi_N$:

$$DK(N_0, C) = \frac{1}{n!} \sum_{\pi \in \Pi_N} DK^\pi(N_0, C),$$

where $DK^\pi(N_0, C)$ is the allocation obtained by applying Prim's algorithm to (N_0, C) and solving indifference by selecting the first agent given by the permutation π .

Value

A list containing:

- dk: the Dutta-Kar's allocation vector $DK(N_0, C)$ for the natural ordering.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- e_dk: extended Dutta-Kar's allocation (average over permutations).
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- nperms: the number of permutations $n!$ computed for the average allocation (if ties exist).
- perms: a data frame with the detailed allocations for all permutations (if ties exist).
- is_unique: logical; if TRUE, the MCST is unique.

Note

The function identifies if ties exist in the cost matrix. If the tree is unique, the standard Dutta-Kar rule is returned. If ties are detected, it calculates the allocations for all $n!$ permutations to provide the symmetrical average allocation, as discussed in Bergantiños and Vidal-Puga (2021).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Dutta B, Kar A (2004) Cost monotonicity, consistency and minimum cost spanning tree games. *Games Econom Behav* 48(2):223–248

See Also

[bird_rule](#) for other rule based on Prim's algorithm.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
dk_rule(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
dk_rule(C_inf)

# Matrix input with ties
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
dk_rule(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

folk_rule

*Folk Rule for MCSTP***Description**

This function computes folk cost allocation (Feltkamp et al., 1994) for a minimum cost spanning tree problem (N_0, C) . The folk rule, denoted as $F(N_0, C)$, also known as the ERO (Equal Remaining Obligation) rule, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
folk_rule(C, draw = FALSE, titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The folk rule is based on the obligation function:

$$o_i(S) = \frac{1}{|S|}.$$

At each stage p , let $c_{i^p j^p}$ be the cost of the arc added to the network. The folk allocation for each agent $i \in N$ is given by:

$$F_i(N_0, C) = \sum_{p=1}^n c_{i^p j^p} \left(o_i(S_i^{p-1}) - o_i(S_i^p) \right).$$

Value

A list containing:

- folk: the folk allocation vector $F(N_0, C)$.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- is_unique: logical; if TRUE, the MCST is unique.

Note

The folk allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Feltkamp V, Tijs S, Muto S (1994) On the irreducible core and the equal remaining obligations rule of minimum cost spanning extension problems. Technical Report 106, CentER DP 1994, Tilburg University, The Netherlands

See Also

[ows_rule](#), [pws_rule](#) for other rules based on Kruskal's algorithm.

[boruvka_rule](#) for an equivalent rule based on Boruvka's algorithm.

[conewise_rule](#) with rule = "folk" for the equivalent implementation based on cone-wise decomposition.

[irred_game](#), [opt_game](#) for cooperative games whose Shapley value coincides with the folk solution.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
folk_rule(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
folk_rule(C_inf)

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
folk_rule(C_mat, draw = TRUE, titles = FALSE)
```

irred_game

Cost Allocations based on the Irreducible Game for MCSTP

Description

This function computes the cost allocation associated with the irreducible game (Bird, 1976) for a minimum cost spanning tree problem (N_0, C) . The irreducible game, denoted as (N, v_C^i) (or simply (N, v^i)), is defined as the private game associated with the irreducible cost matrix $\bar{C}$. When the Shapley value is applied as the solution concept to this game, the resulting allocation coincides with the folk rule of (N_0, C) , and Bird's rule of $(N_0, \bar{C})$ (Bergantiños and Vidal-Puga, 2007a).

Usage

```
irred_game(C, sol = "shapley", draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
sol	character; the solution concept to apply to the game. Currently, only sol = "shapley" (default) is supported.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$, it also displays the optimal network for each subcoalition $S \subset N$.
which	numeric vector indicating which plots to display. If 1 (only for $n \leq 3$), displays the optimal network for each subcoalition $S \subset N$; if 2, the allocation for N is plotted. Default is c(1, 2).
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept used and the total network cost.

Details

The irreducible game (N, v_C^i) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^i(S) = m(S_0, \bar{C}),$$

where $m(S_0, \bar{C})$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, evaluated over the irreducible costs $\bar{C}$. The irreducible cost between any two nodes corresponds to the maximum cost arc on the unique path connecting them in the optimal tree.

Applying the Shapley value to this TU game assigns to each agent $i \in N$ the average of their marginal contributions over all possible permutations $\pi \in \Pi_N$:

$$Sh_i(N, v^i) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v^i(Pre(i, \pi) \cup \{i\}) - v^i(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$ ordered by cardinality.
- **C_irred**: the irreducible cost matrix $\bar{C}$ associated with the MCST of the problem.
- **v_i**: a vector containing the characteristic function values $v^i(S)$ for all coalitions.
- **contributions**: a data frame with the marginal contributions for all permutations $\pi \in \Pi_N$.
- **shapley**: the Shapley value allocation vector.
- **total**: the total cost of the MCST, $v^i(N)$.

- `nperms`: the number of permutations $n!$ used to compute marginal contributions and the resulting Shapley value.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: ranking of agents by cost (from highest to lowest; ties marked with *).

Note

The irreducible game is a strong reference for stability. Its core (the irreducible core) is always non-empty and constitutes a subset of the core of v^p . Classical rules such as Bird and folk are known to belong to the core of v^i .

Unlike the private game, calculating solution concepts for the irreducible game is computationally efficient. It has been proven that computing both the Shapley value and the nucleolus of v^i can be done in $O(|N|^2)$ time.

References

- Bergantiños G, Vidal-Puga J (2007a) A fair rule in minimum cost spanning tree problems. *J Econom Theory* 137(1):326–352
- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335–350

See Also

`bird_rule` of $(N_0, \bar{C})$, `folk_rule` of (N_0, C) for classical rules belonging to the core of v^i that coincide with $Sh(N, v^i)$.

`private_game`, `public_game` for the pessimistic and optimistic games based on the original costs.

`cc_game` for the pessimistic game based on the cycle-complete cost matrix C^* .

`opt_game` for an equivalent game under the Shapley value.

`alloc_rules` for an overview of the available rules in the package.

Examples

```
# Simple vector input
irred_game(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
irred_game(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
irred_game(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

opt_game

*Cost Allocations based on the Optimistic Game for MCSTP***Description**

This function computes the cost allocation associated with the optimistic game (Bergantiños and Vidal-Puga, 2007b) for a minimum cost spanning tree problem (N_0, C) . The optimistic game, denoted as (N, v_C^o) (or simply (N, v^o)), evaluates the cost of each coalition $S \subseteq N$ assuming that agents in $N \setminus S$ are already connected. Thus, agents in S can connect to the source through agents in $N \setminus S$ for free. When the Shapley value is applied as the solution concept to this game, the resulting allocation coincides with the Shapley value of the irreducible game (N, v_C^i) (Bergantiños and Vidal-Puga, 2007b).

Usage

```
opt_game(C, sol = "shapley", draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
sol	character; the solution concept to apply to the game. Currently, only sol = "shapley" (default) is supported.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$, it also displays the optimal network for each subcoalition $S \subset N$.
which	numeric vector indicating which plots to display. If 1 (only for $n \leq 3$), displays the optimal network for each subcoalition $S \subset N$; if 2, the allocation for N is plotted. Default is c(1, 2).
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept used and the total network cost.

Details

Let $T := N \setminus S$. The optimistic game (N, v_C^o) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^o(S) = m(S_0, C^T),$$

where $m(S_0, C^T)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, using the modified cost matrix C^T . In this matrix, the cost between agents in S remains the same, i.e., $c_{ij}^T = c_{ij}$ for all $i, j \in S$, but the cost to connect any agent $i \in S$ to the source is updated as:

$$c_{0i}^T = \min_{j \in T \cup \{0\}} c_{ji}.$$

Applying the Shapley value to this TU game assigns to each agent $i \in N$ the average of their marginal contributions over all possible permutations $\pi \in \Pi_N$:

$$Sh_i(N, v^o) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v^o(Pre(i, \pi) \cup \{i\}) - v^o(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$ ordered by cardinality.
- **C_opt**: a list of the optimistic cost matrices C^T for each coalition $S \subseteq N$.
- **v_o**: a vector containing the characteristic function values $v^o(S)$ for all coalitions.
- **contributions**: a data frame with the marginal contributions for all permutations $\pi \in \Pi_N$.
- **shapley**: the Shapley value allocation vector.
- **total**: the total cost of the MCST, $v^o(N)$.
- **nperms**: the number of permutations $n!$ used to compute marginal contributions and the resulting Shapley value.
- **percentage**: the share of the total cost allocated to each agent.
- **ranking**: ranking of agents by cost (from highest to lowest; ties marked with *).

Note

The optimistic game associated with any MCST problem coincides with the optimistic game associated with its irreducible form, i.e.,

$$v_C^o = v_{\bar{C}}^o.$$

If $(N_0, \bar{C})$ is irreducible, the private and optimistic games, v^p and v^o , are dual: $v^p(S) + v^o(N \setminus S) = m(N_0, C)$ for all $S \subset N$.

Furthermore, as $Sh(N, v_C^o) = Sh(N, v_C^i)$, applying the Shapley value to v^o is simply another way of obtaining the [folk_rule](#) (Bergantiños and Vidal-Puga, 2007b).

References

Bergantiños G, Vidal-Puga J (2007b) The optimistic TU game in minimum cost spanning tree problems. *Int J Game Theory* 36(2):223–239

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[private_game](#) for a game based on a pessimistic approach.

[irred_game](#) for an equivalent game under the Shapley value.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
opt_game(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
opt_game(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
opt_game(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

ows_rule

Optimistic Weighted Shapley's Rule for MCSTP

Description

This function computes the optimistic weighted Shapley cost allocation (Bergantiños and Lorenzo-Freire, 2008a,b) for a minimum cost spanning tree problem (N_0, C) . The optimistic weighted Shapley rule, denoted as $f^{\ell^{ow}}(N_0, C)$, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
ows_rule(C, weights, draw = FALSE, titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The optimistic weighted Shapley rule is based on the obligation function:

$$o_i(S) = \frac{w_i}{\sum_{j \in S} w_j},$$

where $w_i > 0$ is the weight associated with agent i .

At each stage p , let c_{ij}^p be the cost of the arc added to the network. The optimistic weighted Shapley allocation for each agent $i \in N$ is given by:

$$f_i^{\ell^{ow}}(N_0, C) = \sum_{p=1}^n c_{ij}^p \left(o_i(S_i^{p-1}) - o_i(S_i^p) \right).$$

Value

A list containing:

- `ows`: the optimistic weighted Shapley allocation vector $f^{\ell^{ow}}(N_0, C)$.
- `weights`: the weight vector used for the allocation.
- `arcs`: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- `total`: the total cost of the MCST, $m(N_0, C)$.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: a ranking of agents by cost (from highest to lowest; ties marked with *).
- `is_unique`: logical; if TRUE, the MCST is unique.

Note

The optimistic weighted Shapley allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

Furthermore, when all agents share the same weight, the optimistic weighted Shapley allocation is mathematically equivalent to the [folk_rule](#).

References

- Bergantiños G, Lorenzo-Freire S (2008a) A characterization of optimistic weighted Shapley rules in minimum cost spanning tree problems. *Econ Theor* 35(3):523–538
- Bergantiños G, Lorenzo-Freire S (2008b) “Optimistic” weighted Shapley rules in minimum cost spanning tree problems. *European J Operatl Res* 185(1):289–298
- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[folk_rule](#), [pws_rule](#) for other rules based on Kruskal's algorithm.

[conewise_rule](#) with `rule = "ows"` for the equivalent implementation based on cone-wise decomposition.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input with custom weights
ows_rule(c(12, 15, 20, 4, 6, 8), weights = c(2, 1, 1))

# Matrix input with equal weights (equivalent to folk rule)
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
ows_rule(C_mat, weights = rep(1,3), draw = TRUE, titles = FALSE)
```

private_game

Cost Allocations based on the Private Game for MCSTP (Kar's Rule)

Description

This function computes the cost allocation associated with the private game, also known as the pessimistic game (Bird, 1976), for a minimum cost spanning tree problem (N_0, C) . The private game, denoted as (N, v_C^p) (or simply (N, v^p)), evaluates the cost of each coalition $S \subseteq N$ under the assumption that agents outside the coalition are not available to provide their nodes as intermediaries. When the Shapley value is applied as the solution concept to this game, the resulting allocation is known as the Kar's rule (Kar, 2002).

Usage

```
private_game(C, sol = "shapley", draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
sol	character; the solution concept to apply to the game. Currently, only sol = "shapley" (default) is supported.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$, it also displays the optimal network for each subcoalition $S \subset N$.
which	numeric vector indicating which plots to display. If 1 (only for $n \leq 3$), displays the optimal network for each subcoalition $S \subset N$; if 2, the allocation for N is plotted. Default is c(1, 2).
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept used and the total network cost.

Details

The private game (N, v_C^p) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^p(S) = m(S_0, C),$$

where $m(S_0, C)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source. This approach is termed "private" or "pessimistic" because the nodes in $N \setminus S$ belong to these agents, and their participation is needed in order to use their nodes.

Kar's rule is defined by applying the Shapley value to this TU game, which assigns to each agent $i \in N$ the average of their marginal contributions over all possible permutations $\pi \in \Pi_N$:

$$Sh_i(N, v^p) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v^p(Pre(i, \pi) \cup \{i\}) - v^p(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

Value

A list containing:

- coalitions: a character vector representing all possible coalitions $S \subseteq N$ ordered by cardinality.
- v_p: a vector containing the characteristic function values $v^p(S)$ for all coalitions.
- contributions: a data frame with the marginal contributions for all permutations $\pi \in \Pi_N$.
- shapley: the Shapley value allocation vector (Kar's rule).
- total: the total cost of the MCST, $v^p(N)$.
- nperms: the number of permutations $n!$ used to compute marginal contributions and the resulting Shapley value.
- percentage: the share of the total cost allocated to each agent.
- ranking: ranking of agents by cost (from highest to lowest; ties marked with *).

Note

The private game is a strong reference for stability. Classical rules such as Bird, Dutta-Kar, or the folk rule are known to belong to the core of v^p for any MCST problem. As a result, these allocations are coalitionally stable under the pessimistic cost perspective.

Beyond the Shapley value, other solution concepts have been studied for this game, including the core and the nucleolus. It has been proven that computing both the Shapley value and the nucleolus is NP-hard.

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335-350
- Kar A (2002) Axiomatization of the Shapley value on minimum cost spanning tree games. *Games Econom Behav* 38(2):265-277

See Also

[bird_rule](#), [dk_rule](#), [folk_rule](#) for classical rules belonging to the core of v^p .
[irred_game](#) for the pessimistic game based on the irreducible cost matrix $\bar{C}$.
[cc_game](#) for the pessimistic game based on the cycle-complete cost matrix C^* .
[opt_game](#), [public_game](#) for optimistic alternatives to the private approach.
[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
private_game(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
private_game(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
private_game(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

public_game

*Cost Allocations based on the Public Game for MCSTP***Description**

This function computes the cost allocation associated with the public game (Bogomolnaia and Moulin, 2010) for a minimum cost spanning tree problem (N_0, C) . The public game, denoted as (N, v_C^u) (or simply (N, v^u)), evaluates the cost of each coalition $S \subseteq N$ assuming that there are no property rights on nodes. Therefore, agents in S can use the nodes of agents in $N \setminus S$ to connect to the source, paying the corresponding arc costs.

Usage

```
public_game(C, sol = "shapley", draw = FALSE, which = c(1, 2), titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
sol	character; the solution concept to apply to the game. Currently, only sol = "shapley" (default) is supported.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$, it also displays the optimal network for each subcoalition $S \subset N$.

which	numeric vector indicating which plots to display. If 1 (only for $n \leq 3$), displays the optimal network for each subcoalition $S \subset N$; if 2, the allocation for N is plotted. Default is <code>c(1, 2)</code> .
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept used and the total network cost.

Details

The public game (N, v_C^u) is a cooperative game with transferable utility (TU game) where the cost for each coalition $S \subseteq N$ is defined as the minimum private game cost among all its supersets $T \supseteq S$, i.e.:

$$v_C^u(S) = \min_{S \subseteq T} v_C^p(T),$$

where $v_C^p(\cdot)$ is the characteristic function of the private game. This means computing $v^u(S)$ requires calculating the private game costs for all possible supersets of S .

Applying the Shapley value to this TU game assigns to each agent $i \in N$ the average of their marginal contributions over all possible permutations $\pi \in \Pi_N$:

$$Sh_i(N, v^u) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v^u(Pre(i, \pi) \cup \{i\}) - v^u(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

Value

A list containing:

- `coalitions`: a character vector representing all possible coalitions $S \subseteq N$ ordered by cardinality.
- `v_u`: a vector containing the characteristic function values $v^u(S)$ for all coalitions.
- `v_p`: a vector containing the characteristic function values $v^p(S)$ for all coalitions.
- `best_T`: a data frame mapping each coalition S to the optimal superset T that minimizes its cost.
- `contributions`: a data frame with the marginal contributions for all permutations $\pi \in \Pi_N$.
- `shapley`: the Shapley value allocation vector.
- `total`: the total cost of the network, $v^u(N)$.
- `nperms`: the number of permutations $n!$ used to compute marginal contributions and the resulting Shapley value.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: ranking of agents by cost (from highest to lowest; ties marked with *).

Note

The core of v^u coincides with the core of v^p restricted to nonnegative cost shares. Consequently, classical rules such as Bird, Dutta-Kar, or the folk rule belong to the core of the public game for any MCST problem.

However, unlike in the private game, the Shapley value of the public game does not necessarily belong to the core of v^u (Trudeau and Vidal-Puga, 2019).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Bogomolnaia A, Moulin H (2010) Sharing a minimal cost spanning tree: beyond the folk solution. *Games and Econom Behav* 69(2):238–248
- Trudeau C, Vidal-Puga J (2019) The Shapley value in minimum cost spanning tree problems. In: Algaba E, Fragnelli V, Sánchez-Soriano J (eds) *Chapters in Game Theory: The Shapley value*, chapter 24. CRC Press, Taylor & Francis Group

See Also

[bird_rule](#), [dk_rule](#), [folk_rule](#) for classical rules belonging to the core of v^u .

[private_game](#) for the pessimistic alternative to the public game.

[irred_game](#) for a game that can be defined from either the private or the public approach.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input
public_game(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
public_game(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
public_game(C_mat, draw = TRUE, which = 2, titles = FALSE)
```

pws_rule

Pessimistic Weighted Shapley's Rule for MCSTP

Description

This function computes the pessimistic weighted Shapley cost allocation (Lorenzo and Lorenzo-Freire, 2009) for a minimum cost spanning tree problem (N_0, C) . The pessimistic weighted Shapley rule, denoted as $f^{p^w}(N_0, C)$, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
pws_rule(C, weights, draw = FALSE, titles = TRUE)
```

Arguments

C	a symmetric square matrix or a numeric vector representing the lower triangle of costs (ordered by columns) among the nodes in N_0 . The first row and column are assumed to be the source (0). Supports Inf for disconnected nodes.
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The pessimistic weighted Shapley rule is based on the obligation function:

$$o_i(S) = \sum_{\pi \in \Pi(S \setminus \{i\})} \prod_{j=1}^{|S|-1} \frac{w_{\pi^{-1}(j)}}{\sum_{k=1}^j w_{\pi^{-1}(k)} + w_i},$$

where $\Pi(\cdot)$ denotes the set of all permutations of a given set, and $w_i > 0$ is the weight associated with agent i .

At each stage p , let $c_{i^p j^p}$ be the cost of the arc added to the network. The pessimistic weighted Shapley allocation for each agent $i \in N$ is given by:

$$f_i^{p^w}(N_0, C) = \sum_{p=1}^n c_{i^p j^p} (o_i(S_i^{p-1}) - o_i(S_i^p)).$$

Value

A list containing:

- pws: the pessimistic weighted Shapley allocation vector $f^{p^w}(N_0, C)$.
- weights: the weight vector used for the allocation.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- is_unique: logical; if TRUE, the MCST is unique.

Note

The pessimistic weighted Shapley allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

Furthermore, when all agents share the same weight, the pessimistic weighted Shapley allocation is mathematically equivalent to the [folk_rule](#).

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Lorenzo L, Lorenzo-Freire S (2009) A characterization of Kruskal sharing rules for minimum cost spanning tree problems. *Int J Game Theory* 38(1):107–126

See Also

[folk_rule](#), [ows_rule](#) for other rules based on Kruskal's algorithm.

[alloc_rules](#) for an overview of the available rules in the package.

Examples

```
# Simple vector input with custom weights
pws_rule(c(12, 15, 20, 4, 6, 8), weights = c(2, 1, 1))

# Matrix input with equal weights (equivalent to folk rule)
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
pws_rule(C_mat, weights = rep(1,3), draw = TRUE, titles = FALSE)
```

Index

* Algorithmic Rules

- alloc_rules, 2
- bird_rule, 3
- boruvka_rule, 5
- conewise_rule, 9
- dk_rule, 12
- folk_rule, 14
- ows_rule, 20
- pws_rule, 26

* Cooperative Games

- alloc_rules, 2
- cc_game, 7
- irred_game, 15
- opt_game, 18
- private_game, 22
- public_game, 24

* MCSTP

- alloc_rules, 2
- bird_rule, 3
- boruvka_rule, 5
- cc_game, 7
- conewise_rule, 9
- dk_rule, 12
- folk_rule, 14
- irred_game, 15
- opt_game, 18
- ows_rule, 20
- private_game, 22
- public_game, 24
- pws_rule, 26

irred_game, 3, 5, 9, 15, 15, 19, 24, 26

opt_game, 3, 15, 17, 18, 24

ows_rule, 2, 11, 15, 20, 28

private_game, 3, 9, 17, 19, 22, 26

public_game, 3, 17, 24, 24

pws_rule, 2, 15, 21, 26

alloc_rules, 2, 5, 7, 9, 11, 13, 15, 17, 19, 21,
24, 26, 28

bird_rule, 2, 3, 13, 17, 24, 26

boruvka_rule, 2, 5, 15

cc_game, 3, 7, 17, 24

conewise_rule, 2, 9, 15, 21

dk_rule, 2, 5, 12, 24, 26

folk_rule, 2, 6, 7, 11, 14, 17, 19, 21, 24,
26–28